

Tarefa 16: Produto Matriz-Vetor Usando MPI

Distribuição com MPI_Scatter e MPI_Bcast e coleta com MPI_Gather

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Setembro de 2026

1. Objetivo

Implementar um programa MPI que calcule o produto $y = A \cdot x$, em que A é uma matriz $M \times N$ e x é um vetor de tamanho N . A matriz A deve ser dividida por linhas entre os processos com MPI_Scatter e o vetor x distribuído inteiro com MPI_Bcast. Cada processo calcula os elementos de y correspondentes às suas linhas e os envia de volta ao processo 0 com MPI_Gather. Comparar os tempos com diferentes tamanhos de matriz e números de processos.

2. Ambiente de Execução

Item	Valor
Máquina	Supercomputador NPAD/UFRN, partição amd-512, nó exclusivo
Nó	r2n09 — 128 núcleos, de 1 a 128 processos no mesmo nó
Processador	2 × AMD EPYC 7713 (Milan)
MPI	OpenMPI 5.0.10 (libraries/openmpi/5.0.10-gnu14-ucxmt)
Compilador	GCC 14.2.0 via mpicc -O2
Job	SLURM 2127793

3. Implementação

O processo 0 cria A (com $A[i][j] = i + j$) e x (com todos os valores iguais a 1). Com P processos, cada um recebe M/P linhas consecutivas de A por MPI_Scatter (por isso M deve ser múltiplo de P) e o vetor x inteiro por MPI_Bcast. Cada processo calcula as M/P posições de y das suas linhas, e o MPI_Gather junta esses pedaços, na ordem dos processos, no vetor y do processo 0.

O processo 0 mede com MPI_Wtime três trechos: **distribuição** (MPI_Scatter + MPI_Bcast), **cálculo** das suas linhas e **coleta** (MPI_Gather). Ao final, ele confere cada $y[i]$ com o valor exato $N \cdot i + N(N-1)/2$; todas as execuções deram 0 erros.

Foram usadas matrizes quadradas de 1024, 4096 e 16384 (8 MB, 128 MB e 2 GB) com 1, 2, 4, ..., 128 processos. Cada caso rodou 3 vezes e foi usada a execução de menor tempo total; a variação entre repetições foi pequena.

4. Resultados

Matriz 1024 × 1024

Processos	Total (ms)	Distribuição (ms)	Cálculo (ms)	Coleta (ms)	Speedup total	Speedup cálculo
1	2,21	1,20	1,001	0,002	1,00×	1,0×
2	3,51	2,98	0,512	0,021	0,63×	2,0×
4	3,94	3,66	0,264	0,017	0,56×	3,8×
8	3,99	3,63	0,133	0,231	0,55×	7,5×
16	1,45	1,06	0,080	0,309	1,52×	12,5×
32	1,53	1,23	0,044	0,259	1,44×	22,8×
64	2,16	1,84	0,017	0,306	1,02×	58,9×
128	7,54	6,69	0,008	0,834	0,29×	125,1×

Matriz 4096 × 4096

Processos	Total (ms)	Distribuição (ms)	Cálculo (ms)	Coleta (ms)	Speedup total	Speedup cálculo
1	29,99	13,28	16,705	0,008	1,00×	1,0×
2	28,96	20,51	8,413	0,039	1,04×	2,0×
4	28,57	24,13	4,317	0,120	1,05×	3,9×
8	27,22	23,65	3,467	0,105	1,10×	4,8×
16	17,73	13,37	3,473	0,886	1,69×	4,8×
32	13,10	11,24	0,806	1,056	2,29×	20,7×
64	12,43	11,26	0,456	0,713	2,41×	36,6×
128	20,09	18,67	0,130	1,282	1,49×	128,5×

Matriz 16384 × 16384

Processos	Total (ms)	Distribuição (ms)	Cálculo (ms)	Coleta (ms)	Speedup total	Speedup cálculo
1	474,83	205,97	268,844	0,019	1,00×	1,0×
2	428,89	294,17	134,622	0,094	1,11×	2,0×
4	413,87	345,42	67,770	0,687	1,15×	4,0×
8	348,66	295,14	51,725	1,798	1,36×	5,2×
16	230,53	179,59	49,384	1,553	2,06×	5,4×
32	146,18	119,20	25,870	1,114	3,25×	10,4×
64	111,62	96,04	13,749	1,827	4,25×	19,6×
128	129,17	120,49	8,021	0,657	3,68×	33,5×

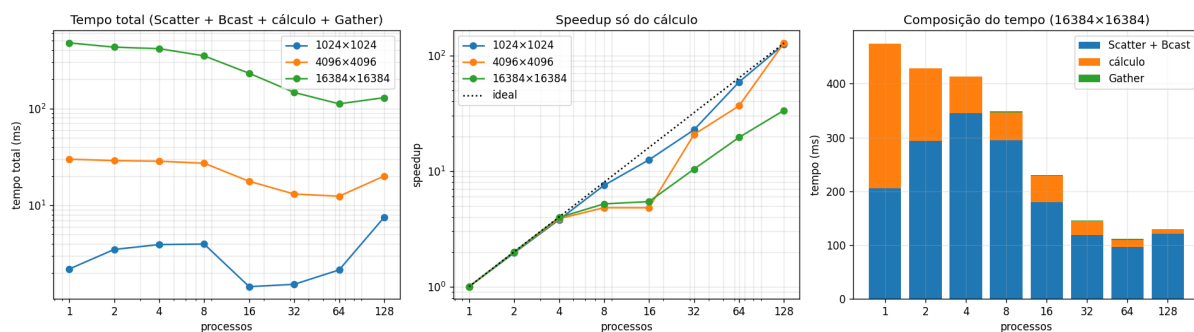


Figura 1 — Tempo total (esquerda), speedup só do cálculo (centro) e composição do tempo para a matriz 16384×16384 (direita), em função do número de processos.

5. Análise

O cálculo escala bem. Na matriz de 16384×16384, o cálculo cai de 269 ms com 1 processo para 8,0 ms com 128 (speedup de 34×), e na de 1024×1024 o speedup do cálculo fica próximo do ideal. Cada processo trabalha só nas suas linhas, sem depender dos outros. Entre 8 e 16 processos o cálculo do processo 0 quase não caiu nas duas matrizes maiores; como o laço é limitado pela memória, isso provavelmente vem da forma como os processos foram posicionados nos domínios NUMA (não foi investigado).

A distribuição domina o tempo total. O produto faz só 2 operações por elemento de A, mas cada elemento precisa sair do processo 0 pelo MPI_Scatter. Na matriz de 16384×16384, a distribuição é 43% do tempo com 1 processo e 93% com 128. Por isso o speedup total é bem menor que o do cálculo: no máximo 4,25× (com 64 processos). Enviar os dados custa mais que calcular com eles.

O tempo de distribuição ainda cai de ~345 ms (4 processos) para ~96 ms (64): no mesmo nó, cada processo copia o seu bloco da memória do processo 0 e escreve no seu próprio vetor, e essas cópias (e o primeiro acesso às páginas novas) acontecem em paralelo. Com 1 processo, a “distribuição” é só a cópia da matriz inteira para o vetor local.

Matrizes pequenas não compensam. Na matriz de 1024×1024 o cálculo leva ~1 ms e o custo fixo das operações coletivas é da mesma ordem: com 2 a 8 processos o programa fica mais lento que com 1, e o melhor caso (16 processos) é só 1,5× mais rápido. Na de 4096×4096 o ganho máximo é 2,4×.

Com 128 processos o tempo volta a subir nos três tamanhos: o cálculo por processo já é pequeno, e a distribuição e a coleta, com mais participantes, ficam mais caras. Ainda assim a coleta é sempre pequena (no máximo ~1 ms), porque y tem só M valores.

6. Conclusão

A divisão por linhas com MPI_Scatter, MPI_Bcast e MPI_Gather dá o resultado correto e o cálculo escala quase linearmente com o número de processos. O tempo total, porém, é dominado por espalhar a matriz a partir do processo 0, porque o produto matriz-vetor faz pouca conta por dado transferido. O ganho só aparece em matrizes grandes e com um número moderado de processos (até 4,3× com 64 processos na maior matriz). Na prática, compensaria cada processo gerar ou ler as suas próprias linhas, em vez de recebê-las do processo 0.

7. Código-Fonte

```
1  #include <mpi.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  int main(int argc, char **argv) {
6      int rank, size;
7      MPI_Init(&argc, &argv);
8      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9      MPI_Comm_size(MPI_COMM_WORLD, &size);
10
11     int m = atoi(argv[1]), n = atoi(argv[2]);
12     int linhas = m / size;                               /* linhas por processo */
13
14     double *A = NULL, *y = NULL;
15     double *x = malloc(n * sizeof(double));
16     double *a = malloc((size_t)linhas * n * sizeof(double)); /* linhas locais */
17     double *y1 = malloc(linhas * sizeof(double));
18
19     if (rank == 0) {                                     /* A[i][j] = i + j, x = 1 */
20         A = malloc((size_t)m * n * sizeof(double));
21         y = malloc(m * sizeof(double));
22         for (int i = 0; i < m; i++)
23             for (int j = 0; j < n; j++) A[(size_t)i * n + j] = i + j;
24         for (int j = 0; j < n; j++) x[j] = 1.0;
25     }
26
27     double t0 = MPI_Wtime();
28     MPI_Scatter(A, linhas * n, MPI_DOUBLE, a, linhas * n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
29     MPI_Bcast(x, n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
30     double t1 = MPI_Wtime();
31
32     for (int i = 0; i < linhas; i++) {
33         double s = 0.0;
34         for (int j = 0; j < n; j++) s += a[(size_t)i * n + j] * x[j];
35         y1[i] = s;
36     }
37     double t2 = MPI_Wtime();
38
39     MPI_Gather(y1, linhas, MPI_DOUBLE, y, linhas, MPI_DOUBLE, 0, MPI_COMM_WORLD);
40     double t3 = MPI_Wtime();
41
42     if (rank == 0) {                                     /* y[i] = n*i + n(n-1)/2 */
43         int erros = 0;
44         for (int i = 0; i < m; i++)
45             if (y[i] != (double)n * i + (double)n * (n - 1) / 2) erros++;
46         printf("%d,%d,%d,%.6f,%.6f,%.6f,%.6f,%.6f,%d\n", m, n, size,
47                t3 - t0, t1 - t0, t2 - t1, t3 - t2, erros);
48         free(A);
49         free(y);
50     }
51     free(x);
52     free(a);
53     free(y1);
54     MPI_Finalize();
55     return 0;
56 }
57
```

Figura 2 — mxv.c.